Department of Computer Science

Summative Coursework Front Page

Module Title	Programming in Python
Module Code	CS2PP
Lecturer Responsible	Dr Todd Jones
Assignment Title & Description	Python Application Group Project QR Application – CLI, GUI, and Extensions
Type of Assignment	Set Exercise
Learning Outcomes Assessed	1. Implement algorithms to design solutions to programming problems with Python scripts and software applications. 2. Assemble code that incorporates imperative, functional, and object-oriented programming paradigms. 3. Integrate third-party Python libraries to formulate and construct Python programs for practical applications. 4. Demonstrate best practices and conventions for writing clean and efficient code.
Individual or Group Assignment	Group
Weighting of the Assignment	60%
Eligible for Self-Certification Extension	No
Word count/Page limit	1500 words (Excluding code and comments; primarily documentation and reflection work)
Typical expected hrs spent for the assignment	24 hours per group member
Items to be submitted	In-person demonstration, 16/17 December. A single **`.zip`** archive of one **csgitlab** repository. Required repository contents detailed below. Submission Notes on Blackboard.
Work to be submitted on-line via **Gradescope** by	**2027 January 18th (Monday) 12:00 (noon)**
Work will be marked and returned by	Within 15 working days of due date or late submission
Artificial Intelligence Tools	May be used to support work

Note

By submitting this work, you are certifying that you have read the assessment guidelines displayed in the Assessment folder on the Blackboard course for this module, and that you have understood and complied with the associated policies and practices, including those relating to:

- submitting your own work, rather than work produced by other people or systems (including those using artificial intelligence), and the associated penalties for Academic Misconduct;
- submitting your work by the specified deadline and the penalties associated with late submission, where applicable;
- complying with the requirements of the Assessment Adjustments system, where applicable;
- and attaching a green sticker for relevant needs to your submission to indicate that you have an approved adjustment.

1. Assignment Description

This is a **group assessment with individually assessed components**. Assessment incorporates **code, documentation, and reflective submissions, as well as in-person demonstrations.** Multiple elements will be used to assess your implementation of several aspects of Python by harnessing third-party tools.

A detailed breakdown of the Marking Scheme is provided later in this document.

Project Demonstration: Week 12 Practical sessions: 16 & 17 December 2026, 09:00-11:00 in the Polly Vacher Building - on lab PCs

Code and Documentation Submission: Monday 18th January 12:00 (noon)

Group Formation

Groups will ideally have **5 members.** A few might have 4.

Groups should be formed of those who are assigned to the same timetabled practical session (A or B; Wednesday or Thursday). This will facilitate more straightforward interaction for formative in-person demonstrations of work in progress.

Groups will be available for self-selection via Blackboard from **Monday 19th October**, but you can also make a request to be randomly assigned to a group by emailing t.r.jones@reading.ac.uk if you have not already requested this from the class survey in Week 1. Students with registered accommodations have been contacted about adjustments to this plan.

If you have not selected a group by **Thursday, 12th November at noon**, you will be randomly assigned to a group.

If your group has fewer than 5 members on **Thursday, 12th November at noon**, additional members will be assigned to your group, unless the numbers are uneven, in which case some groups will have 4 members.

Embracing Collaborative Learning and Practical Application

Computer science is a dynamic field where the ability to collaborate effectively and apply theoretical knowledge to real-world problems is invaluable. This group coursework is designed not only to assess your **technical proficiency in Python programming** but also to enhance your **collaborative skills** (which are both social and technical in this context), which are highly valued in the professional world. Research indicates that employers and graduates alike place significant importance on teamwork and the ability to work effectively in diverse groups (Hughes & Jones, 2011; Salas et al., 2005). Through this project, you will demonstrate your understanding of both functional and object-oriented programming paradigms in Python. By engaging in this group task, you will experience firsthand the challenges and rewards of collaborative problem-solving, a critical skill in today's interconnected and constantly changing work environments.

Key Requirements

Your group will collaboratively build a Python application that generates, from scratch, a functional QR code from user-provided text (e.g., a short URL or message) and that logs and displays the resulting scannable QR code via interactive CLI and GUI or website. Then, you will individually construct extensions to that code system to enhance the generation and display functionality.

Use third-party libraries consistent with the module's base conda environment, those specified in this document, those explicitly explored in live sessions, or those explicitly approved by the Lecturer Responsible following formal request.

The following requirements will guide successful completion of the coursework:

0. Marking

- Markers will **NOT** execute your code.
- Marks will be allocated based on your **demonstrations** of functionality alongside the requested code and markdown files.
- In-person demonstrations are required to be presented on teaching lab PCs.
- Time slot assignments for in-person demonstrations will be shared by Friday 20 November and will be stored alongside coursework materials on Blackboard.

1. Basic QR Data Generation (Version 1 complexity) – See the end of this document for a guideline resource.

- Functionality begins by accepting an arbitrary input string (e.g., a short URL or other textual message) dynamically from a user at the command line.
- Encode the string into bits using the QR standard default character set for byte mode.
- Integrate Reed-Solomon error correction at Level L via the **`reedsolo`** library to generate error correction codewords.
- Assemble the bits into the correct 2D matrix. Orient: 0 corresponds to white and 1 to black.
- Include separators, finder, alignment, and timing patterns, the dark module, terminator bits, padding, and format information; project code should generate format strings as a function of error correction level and mask pattern.
- Apply one masking pattern (pattern `0`) to reduce large blocks of the same-coloured modules.
- Ensure the proper QR format is represented in a numeric data array that can be printed to the screen.
- Your solution should include use of the **`numpy`** library.
- Demonstrate processing of the following five strings (no leading or trailing whitespace):

1	`known`
2	`We've succeeded!`
3	`~¡256_-_aA&ñ`
4	`From α to ω...`
5	`Sugarplum_Fairy_Nightmare`

- Intermediate proofs of functionality and correctness should be demonstrated by presentation of output from an implementation of the **`logging`** functionality, with each step clearly annotated. Among other elements, you might show mode and length, bit string, gridded binary, and codeword representations, and you might show the encoded state before and after masking alongside a representation of the mask pattern. If a string fails to complete the encoding process, your code should still report the completion of steps up until the point of failure. Create five **enumerated demonstration files** (e.g., demo_1.txt, demo_2.txt, etc.) to contain this demonstration information for each of the above five strings.

- **This section should be completed and documented before moving on.** It is recommended that you attend the formative demonstration session during the practicals in Week 10.

2. Scannable QR with Interactive Application

- A GUI or Web interface should request text from a user to be converted into a QR code by using the Version 1 implementation developed above, and it should be designed to address potential errors occurring during use.
- Convert the numerical representation of the QR code into an image using (at least) `matplotlib`.
- Provide functionality to save QR image to file.
- Develop an interactive user interface in the form of a GUI or website based on the third-party tools explored in the module sessions (i.e., not Tkinter).
- Ensure the design complies with data protection principles and warns users of potential risks (e.g., phishing scams) in the creation and use of QR codes.
- An "**Information"** utility should be implemented to explain to users how you have modelled information, managed user inputs, and ensured data integrity and security (e.g., sanitisation, error handling, providing user instructions). The method of implementation is up to you. Show its use.
- Demonstrate that the interface can display QR codes in their most basic visual form (black and white module squares) on a computer screen. These must be capable of successful decoding by a typical mobile phone-based QR scanning application.
- **This section should be completed and documented before moving on.**

3. Project Enhancements

Building on the work completed in stages 1 and 2, each member of your group will **uniquely select one** of the following project enhancement activities to implement. Your selected development should be done primarily on your own in your own repository branch in a separate module file.

However, each enhancement might require modifications to other existing or developing code to function cohesively. Continue communication with your group to avoid clashes and to create structures that support the goals of others in your group. Where modifying code outside your own module file, explicitly note authorship in the code.

Full functionality should be evidenced in the individual markdown file (see below).

For the development of this stage, configure GitLab Continuous Integration (CI) that includes linting and at least three (non-trivial) unit tests that are specific to this interactive stage of the project. **Link to the unit tests in your individual reflection and demonstration Markdown file.** All jobs in the pipeline should be shown to succeed.

Each enhancement will later be merged into the main repository branch to work together in a single application (see below).

- Extend the Version 1 Level-L encoding to enable flexible creation of QR codes at Versions 1 & 2 and Levels L, M, Q, and H. This will, by default, include automatic determination of the lowest level of error correction that can fit the user's data. Additionally, provide Version and Level selection options to the user, supplying warnings where appropriate and notification about capacity for the selected combination. Log QR version and ECC level characteristics and report them to the user.

- Apply all eight masking patterns and evaluate them with all four penalty scores to determine the best masking pattern. The best pattern should be used by default. Additionally, provide a mask selection option to the user. Log the final mask pattern and penalty scores and report them to the user.
- Implement an option to enable explanatory display of the creation of a user-supplied QR code in a visual, annotated, step-by-step, slideshow-like presentation to foster digital literacy. This should explain the process of how the QR components are constructed and modified on the way to the final QR visualisation. Ideally, the user will be supplied with manual control of the stepping through the presentation, and they will have the option to save a copy of any visualised step as an image file.
- Enable the user to select inclusive, accessible customisation of the QR display while maintaining scan reliability (e.g., colours, module shapes, frames, sizes, image filters, logos). Consider colour vision deficiency, poor eyesight, low screen brightness, older devices, poor QR scanners. Explain pre-set configurations in a text message to the user. Warn the user if selected options will limit ability to view or scan the QR (e.g., poor contrast, colour inversion).
- Introduce a data-logging component that tracks when and how QR codes are generated. Each time a QR is requested, use **pandas** and an external file to record: timestamp, unique ID, text content, version, error correction level, mask pattern, user-selected options, generation success. Given logs of accumulated use, allow the user to inspect a display of simple analytics (e.g., a bar chart of the frequency of ECC level selection or a line chart comparing successful and failed generation attempts over time).
- ***Challenging.*** Implement the reverse process. Given an image of a QR code (e.g., user-loaded to the application), detect the modules, extract the bits, and decode the text. Assume the image is a well-formed standard QR of group-implemented specification (e.g., in the standard perspective, black and white squares).

4. Individual Reflection and Demonstration

- **Individually**, reflect upon the development of your individual Project Enhancement **ONLY**. Include the following itemised elements:

  i. At the top, a **Code Summary including the below entries.** *For example*:

  ```
  **Student ID: 12345678**
  - Branch name: myname
  - Individually created module(s): `qr_mask_selector.py`
  - Link to merge of individual branch to main:
  https://csgitlab.reading.ac.uk/cs2pp-26/group-a01/-
  /commit/b78112a3075bfab5d8605db3d9da6caf9c76071c
  - Key function/method(s): `choose_best_mask(matrix)`
  - Designed to: Select optimal QR mask using penalty scoring
  - Dependencies: Uses my class `PenaltyScorer`, called from
  `generate_qr.py`
  - Not robust to: Empty matrix, non-binary inputs
  ```

  ii. Describe the process of designing your code and the way in which it is instrumental to the functionality of the collective project.

  iii. Provide an example function/method call or instance creation that uses your code elsewhere in the final application and explain the mechanics of how it integrates.

iv. Write a usage example of what you consider to be the most important function or method in your enhancement and describe its expected input/output/action in one sentence.

v. Present one code snippet from your module that you consider to be "idiomatic Python". Explain why you consider it to meet this definition and why you have chosen to implement the functionality in this way, rather than a more generic way of coding.

vi. What issues did you face in the development of this enhancement, and how might you approach development differently if you were to revise this element?

- **Demonstrate the functionality** of your code via embedded and annotated screenshots and/or screen capture as relevant to your enhancement. Written explanations should support this demonstration.
- **Submission (link to file in the group-submitted README.md):**
  i. Each group member should document their individual efforts in their own separate Markdown file (with a unique name containing their student number: e.g., **`individual_12345678.md`**). Link to this file from **README.md**.
    - Include this reflection file in the main branch/archive for submission, or
    - Configure a separate private repository containing your personal reflection and provide a link to your private repository in the main group-submitted **README.md**.

5. Requirements for Collaborative Coding with Individual Contributions

- The project must consist of **collaborative coding efforts, accompanied by implementation of code produced individually.**
- Each member is responsible for the creation of their own **identifiably labelled** Python **module file** containing at least **one function or class** that will be **imported** into the main project application.
- The group code base will reside on **csgitlab.reading.ac.uk** in a repository:
  i. within an assigned sub-group of csgitlab.reading.ac.uk/cs2pp-26 named to correspond to your Blackboard group ID
  ii. visibility set to **Private**
  iii. to which the group members will be added in the **Maintainer** role
- In addition to the **main** branch, all group members will have their **own personal branch**, where they will work to develop their contributions.
- All developments will be merged into the **main** branch in the final version.

6. Group Reflective Account

- **As a group**, provide a **`group_reflection.md`** Markdown file in the main branch containing a written account of the ways in which you worked as a group. To direct deeper introspection, begin by addressing the following questions. In what manner and on which dates did you meet? Which communication strategies did you employ? Who assumed which roles? How were tasks planned? How did you organise code dependencies and merging? What challenges did you face? How would you approach these interactions differently in the future?
  i. Were contributions of effort distributed unequally? If so, express the contributions of effort as a percentage (i.e., 100% is full effort, assumed by default). **See note following Marking Scheme table.**

ii. If a group member did not contribute their own module code or their contribution performs improperly, please note how this affected the implementation.

iii. Report any problems in contacting or interacting with group members to module leaders as soon as possible to prevent development delays.

- **In the reflective elements, specifics are essential! You can read this** reflection guidance **to guide your writing**.

7. The Final Product

- Merge the individual developments from stage 3 back into the main branch.
- The main **README.md** will act as a well-formatted user manual for operating your QR product, well-considered comments on its characteristics, and a place for directing readers to demonstration of various aspects of functionality. It should contain the following sections:

i. Compulsory submission items (see next section).

ii. A description of the final application's functionality and operation instructions to promote clarity and transparency. What must a user do to configure their Python environment and get the application running? What options are available to the user, and what do these do? Support these descriptions with an architecture diagram depicting the structure of your application components, their dependencies, and their inputs/outputs.

iii. A list of the Project Enhancements that have been implemented and their authors.

iv. A summary of the ways in which imperative, functional, and object-oriented programming were used in the collective application. Copy an example code snippet for each of these three paradigms and provide a link to their implementation in the final repository.

v. Outline the specific weaknesses and flaws in your final, merged application. For example, is it susceptible to misuse or potentially unsafe? How? Which features of your application would make it unsuitable for use in support of real-world functions (e.g., marketing, inventory, science, tracking)? Supply file names, function names, line numbers, code snippets, etc. to specifically identify these concerns.

vi. Demonstration of all enhancements working in the final application. This could be effectively conveyed via brief screen recordings that show a launch of the final application and explanatory use of each enhancement. Annotate (in recording or Markdown) to describe what is being shown.

vii. Link to the Group Reflective Account file

viii. Links to Individual Reflection files

ix. Any relevant literature references and a description of the ways artificial intelligence tools were used in the development of this application and its associated written work.

2. Marking Scheme

Element	Marks Available
Group Elements*	**70**
Basic QR Data Generation (in-person demonstration)	**20**
Data Encoding and Error Correction Integration: Correct handling of Byte Mode. Proper mode bits, character count bits, and data bit assembly (terminator + padding). Usage of the `reedsolo` library at ECC Level L. Correct total data codewords vs. ECC codewords for Version 1 (e.g., 19 data + 7 ECC = 26 total). Data + ECC codewords are correctly generated (order and count).	5
Matrix Layout and Patterns: Finder patterns in corners, timing pattern, dark module, format information area reserved. Zigzag placement of the data/ECC bits in the right modules (skipping reserved cells). Accurate application of masking pattern `0`.	**5**
Demonstration and Code: Enables user input. Resulting text-based QR is structurally consistent with Version 1 (21×21) or other expected forms for five strings. Intermediate proofs are shown via `logging` files. Code is well structured, documented, and referenced. Understanding of the presented work is demonstrated. Execution follows direct path from csgitlab repository to Lab PC.	**10**
Scannable QR with Interactive Application (in-person demonstration)	**20**
Basic GUI/Website Interactivity and Design: Including a clear way for the user to enter text/URL and trigger the QR generation. The interface is reasonably user-friendly (labels, instructions, layout, information). Data protection is considered.	**7**
Presentation and Visualisation: The expected QR codes are rendered as a scannable image on the screen.	**3**
Demonstration and Code: Demonstration of requested functionality and tests with appropriate results. Code is well structured, documented, and referenced. Understanding of the presented work is demonstrated. Execution follows direct path from csgitlab repository to Lab PC.	**10**
Final Product Documentation (README.md)	**30**
Structure: Requested elements provided with necessary structure.	**2**
Functionality and Operation: Provide complete responses.	**6**
Programming Paradigms: Provide complete responses.	**3**
Weaknesses and Flaws: Provide complete responses.	**3**
Integration and Quality of Final Application: All project enhancements merged into main branch, demonstrated to be working together (screen capture evidence).	**8**
Group Reflection: Quality of group reflection on teamwork, role distribution, and collaborative process; *include member name/effort weight table (see below).*	**8**

Continues next page…

Individual Elements	**30**
Individual Reflection: Prompt responses. Depth of individual reflection on your contribution, challenges faced, and learning outcomes.	**10**
Development: Three significant unit tests implemented and explained. CI implemented and shown to succeed.	**5**
Individual Module Code and Documentation: Quality and functionality of individual code contributions, clarity of intended scope, code structure, and documentation. Functionality evidenced in individual markdown. Details will vary; refer to requirements.	**15**
Coursework 2 Total	**100**

*These elements are group marks. By default, marks of this type will be applied equally to all members of the group. All members are entitled to retain 25% of the group mark. However, 75% of the group mark may be ***modified by proportional weighting*** in the event contributions of effort toward completion were distributed unequally. **Note effort contributions as percentages in the group reflection Markdown submission within your own copy of the leftmost 2 columns below**.

For example, for a group mark of 60:

Provide in Group Reflection			Marking Calculation Example			
Member Name	**Effort Weight**		Group Mark	Weighted Group Mark	Individual Mark	Total Mark
ABC	**80%**		60	51	9	60
DEF	**50%**		60	37.5	8	45.5
GHI	**0%**		60	15	10	25
JKL	**100%**		60	60	5	65
MNO	**100%**		60	60	10	70

This coursework assesses your ability against the Learning Objectives on the front page of the specification.

In general, you will gain credit for:

- preparing and submitting required files as requested;
- successful implementation of the specified tasks;
- writing efficient, functional code;
- providing thoughtful, clear, well-structured written analysis that conveys complex information understandably.

Your assignment will be marked according to the marking scheme provided above. The scheme is designed so that the collectively weighted assignment mark will generally correspond to the following qualitative degree classification descriptions:

Classification Range	Typically, the work should meet these specifications:
First Class (85 – 100%)	This work demonstrates coding proficiency with high efficiency and based on advanced techniques. Evidence of independent research into the methods used and a thorough justification of applications of these methods is presented clearly. Work at this level demonstrates exceptional understanding, creativity, and application in all aspects of the project. The code is exemplary in quality, showcasing exceptional efficiency, organisation, and innovative techniques; documentation is thorough, enhancing clarity and understanding.
First Class (70 – below 85%)	Very good work with only minor errors or omissions. Work at this level demonstrates a thorough understanding of the project objectives and confident application of appropriate methods. Evidence of independent investigation and sound justification of the approaches used is presented clearly. The code demonstrates a high level of proficiency, with efficient and well-organised implementation; documentation is clear and supports readability, reproducibility, and understanding.
Upper Second (60 – below 70%)	Good work with few mistakes. Some minor tasks have not been carried out or are not completely correct. Work at this level exhibits strong comprehension and application of the project's objectives. The code demonstrates strong proficiency, with efficient and well-organised structure; it is well-documented, facilitating easy readability and comprehension.
Lower Second (50 – below 60%)	Demonstrates knowledge of core concepts but with some notable mistakes or omissions. Work at this level demonstrates a good grasp of the project requirements. The code is well-structured and functional, with a good level of efficiency and readability; documentation is clear and aids understanding.
Third (40 – below 50%)	Some parts of the assignment are missing and/or have partially correct results. Work at this level shows an adequate understanding of the project's scope with satisfactory data collection and analysis. Technical aspects, including Python tool usage, are competently executed but lack sophistication. Individual contributions are notable, and there is evidence of teamwork, though not fully effective. The code is functional but simplistic, with some issues in efficiency and structure; basic documentation is present but lacks detail.
Pass (35 – below 40%)	Work at this level meets a subset of the basic requirements but lacks depth in these. There is a rudimentary understanding of the project's objectives, with elementary data collection and analysis. Technical execution is basic, with minimal use of Python tools. Individual contributions fulfil some basic criteria, and group collaboration is limited but present. The code meets basic functional requirements but is rudimentary, often inefficient, and lacks proper organisation or documentation.
Fail (0 – below 35%)	Work at this level fails to meet the minimum requirements. There is a lack of understanding and application of the key concepts and methodologies related to the project. Data collection and analysis are either incorrect, superficial, or missing. Technical implementation is inadequate, with significant errors or misunderstandings. Group collaboration is poorly demonstrated or not evident. The code is either non-functional, severely flawed, or largely absent, showing a fundamental misunderstanding of basic programming concepts.

3. Assignment submission requirements

"Front page" of the Submission

The following are **compulsory**. Add these items to at the **top of your** group README Markdown file.

Module Code:	CS2PP
Assignment report Title:	Python Application Group Project
Student Numbers:	
Date of submission:	
Actual hrs spent for the assignment:	
Other learning aids used (please list):	
CSGitlab Project Repository URL:	csgitlab.reading.ac.uk/cs2pp-26/…

Format of the Required Work

You must use Python (**version 3.13.9**). Where possible, use the packages consistent with the **base** conda environment in the Anaconda3 distribution used in this module (**2025.12-2**). Provide instructions to implement any deviations.

In timetabled practical sessions:

18 & 19 November – Week 8 – Group Status Check (formative)

- Attend to report on status.

2 & 3 December – Week 10 – Formative Progress Demonstration

- Attend to demonstrate elements of progress to date and receive feedback.

16 & 17 December – Week 12 – Demonstration Assessment

- Attend your timetable-assigned practical session to demonstrate elements of Stage 1: Basic QR Data Generation and Stage 2: Scannable QR with Interactive Application for marking.
- Assigned demonstration time slots will be shared by 20 November.
- Demonstrations will be assessed by configuring and running your application on a Polly Vacher lab PC.

Via Blackboard (Gradescope) Monday 18th January 12:00 (noon):

In **addition to all group and individual project code and documentation files**, the files of which you may structure as you see fit, **the following files** are expected at the top level your submission (and repository):

1. **README.md** – See details in Documentation component, 800-word limit for prompt responses
2. **group_reflection.md –** 300-word limit
3. **individual_12345678.md** – one file for each group member, formatted with your 8-digit student ID number, 400-word limit for prompt responses

You will find the Gradescope submission point on the module's Blackboard page under **Assessment**. Here, you can submit the group repository, as downloaded to a **.zip archive** or via **direct upload from csgitlab**, named to correspond to your Blackboard group ID. For example: **Group-A01.zip**.

Please review the **Submission Notes** on Blackboard to ensure your submission is appropriately linked for all group members.

Code Plagiarism

This coursework is expected to be the result of your own group effort, **not that of other people or systems**. That said, you should indeed work closely with others on this coursework. You may employ pair programming techniques, for instance. Copying whole tutorials, scripts or images from external sources is not permitted. Any material you borrow from other sources **to build upon or to support your arguments** should be **clearly referenced** (use comments to reference elements within Python module files and supply formal literature references at the end of the README.md file); otherwise, use of such material will be treated as plagiarism, which may lead to investigation and subsequent action. Work **inspired by** module materials is permitted, but such material should not be used without significant modification. Artificial intelligence tools may be used to support development of ideas for this work, but their outputs should not be copied and submitted; they are also not necessary to complete this work.

Artificial Intelligence Tools

In this assignment, Artificial Intelligence Tools "May be used to support work". Students may choose to use Generative AI to support their learning and the development of their work. For example, they might use it to brainstorm, ask questions, test an outline, or plan a structure to work.

Generative AI use is **not itself assessed** and the final submission must remain the student's own work.

You **may** use Generative AI to explain concepts, summarise material, explore an unfamiliar topic or support early sense-making.

You **may NOT** use Generative AI to refine work that you have already produced, for example its grammar, tone, clarity, structure, expression or length. Use your own voice and write your own code.

You **may NOT** work iteratively with Generative AI to compare ideas, test interpretations, synthesise material or "co-create" an output.

4. Essential Resources

QR Code Guidelines

Multiple instruction sets are available online. However, you are **strongly advised** to follow Thonky.com's QR Code Tutorial: https://www.thonky.com/qr-code-tutorial/

- **This will take some time to read and understand. Do this early. Ask questions.**

Because we ask you to employ the **`reedsolo`** library for error correction, you will not be asked to implement this encoding manually. However, you will need to manually generate error correction bits for the format string as a separate operation.

You can rely on reference to character capacity and total codeword tables for Level L error correction in Versions 1 and 2. These do not need to be calculated explicitly.

- https://www.thonky.com/qr-code-tutorial/character-capacities
- https://www.thonky.com/qr-code-tutorial/error-correction-table

The above site contents are backed up on the Wayback Machine.

Test early and often:

- www.thonky.com/qrcode (Use advanced options, switch to ISO-8859-1)
- softwarehouse.au/tool/qr-code-generator (Mode is automatic; ensure byte used)
  Encodes the string `'known'`, using Version 1, ECC L, Mask 0, Byte Mode:



  Encodes the string `'known'`, using Version 2, ECC L, Mask 4, Byte Mode:



Merge code branches early and often to stay up to date:

- git:
  - https://docs.gitlab.com/19.4/topics/git/
  - https://about.gitlab.com/images/press/git-cheat-sheet.pdf

    - https://git-scm.com/cheat-sheet
- SSH Keys for GitLab
    - https://docs.gitlab.com/19.5/user/ssh/
- merge:
    - https://www.datacamp.com/tutorial/git-merge
    - https://docs.gitlab.com/19.4/topics/git/merge/
- Markdown:
    - https://docs.gitlab.com/ee/user/markdown.html
    - https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet
- CI:
    - https://docs.gitlab.com/19.4/topics/build_your_application/

Group working spaces:

- Make use of Teams, Discord, Zoom, WhatsApp, etc.
- Polly Vacher Group and UG Work Rooms: G44, G46, G47, G49
- Polly Vacher PC Labs: when not being used for classes (G45 will be frequently available)
- University Study Areas: https://www.reading.ac.uk/library/using-the-library/services/study-areas

5. References

Available as PDFs on the Blackboard CW2 Assessment Space

Hughes, R. L., & Jones, S. K. (2011). Developing and assessing college student teamwork skills. *New Directions for Institutional Research*, *2011*(149), 53–64. https://doi.org/https://doi.org/10.1002/ir.380

Salas, E., Sims, D. E., & Burke, C. S. (2005). Is there a “Big Five” in Teamwork? *Small Group Research*, *36*(5), 555–599. https://doi.org/10.1177/1046496405277134